SDLC ASSESSMENT

DataCentre Allocator

Server Resource Management System

BACKEND	DATABASE	FRONTEND	TESTS
Flask · Python 3.11	MongoDB Atlas	HTML · CSS · JS	103 Passing

V1.1 Base Init

V1.2 Server Mgmt

V1.3 Workload Enhancements

Sriram Selvaperumal · Tactive

github.com/Sriram-Selvaperumal/tactive-assignment

01 OVERVIEW

What is this project?

PROBLEM STATEMENT

Data centres run dozens of workloads across many servers. Without automated resource management, operators manually assign tasks — leading to waste, overloaded servers, and stranded capacity.

This system automates that: register servers, queue workloads, and let

SOLUTION

- **Servers:** Register physical/virtual machines with CPU and RAM capacity
- **Workloads:** Define tasks with resource requirements (CPU cores + RAM MB)
- **Allocator:** Automatically assign workloads to the best available server
- **Lifecycle:** Manage server states — Online, Offline, Maintenance — with auto eviction

SERVER

CPU + RAM

capacity pool

WORKLOAD

CPU + RAM

resource request

ALLOCATION

Workload →

Server assignment

02 TECHNOLOGY STACK

BACKEND

Flask 3.1
Lightweight WSGI web framework

Python 3.11+
Type-annotated, dataclass-driven

python-dotenv
Environment variable management

Werkzeug
WSGI utilities and request handling

DATABASE

MongoDB Atlas
Cloud-hosted NoSQL document store

pymongo 4.10
Official MongoDB Python driver

3 Collections
servers · workloads · allocations

Unique Indexes
Enforce name uniqueness + alloc guard

FRONTEND

Vanilla HTML/CSS
No framework dependencies

Vanilla JS
fetch() API, DOM rendering

SPA Architecture
Single page, zero full reloads

Responsive UI
Clean, minimal, card-based layout

TESTING

pytest 8.3
Test runner and assertion library

pytest-flask
Flask test client integration

103 Tests
All green on final run

Integration tests
Black-box, full-stack, no mocking

03 ARCHITECTURE

Browser (Client)

index.html · style.css · app.js

HTTP / JSON →

Routes Layer

Flask Blueprints · Input validation · Response formatting

calls →

Service Layer

AllocationService · Business rules · Best-Fit algorithm

calls →

Repository Layer

ServerRepo · WorkloadRepo · AllocationRepo

queries →

MongoDB Atlas (Cloud)

datacenter_db · servers · workloads · allocations

04 ALLOCATION ENGINE

Rule 1

ONLINE Only

Only servers with status ONLINE are eligible

Rule 2

CPU Sufficiency

available_cpu ≥ workload.cpu_required

Rule 3

RAM Sufficiency

available_ram ≥ workload.ram_required

Rule 4

Both Required

CPU AND RAM must both pass — not either alone

Rule 5

No Partial Allocation

If ineligible, nothing is written — state unchanged

Rule 6

No Duplicate

Allocated workload cannot be assigned again

Rule 7

Best-Fit Selection

Server with smallest remaining slack is chosen

Rule 8

Resource Accounting

allocated_cpu and allocated_ram updated on server

Rule 9

Status Consistency

Workload transitions PENDING → ALLOCATED

Rule 10

Input Validation

Missing or bad workload_id → 400 Bad Request

Best-Fit Algorithm

SCORING FORMULA

$$score = (available\_cpu - required\_cpu) + (available\_ram - required\_ram)$$

Minimum score wins. Tie-broken by oldest registered server.

WORKED EXAMPLE — Workload needs 4 CPU + 8,192 MB RAM

Server	Avail. CPU	Avail. RAM	Score	Selected?
Server A	16	32,768 MB	24,588	—
Server B	8	16,384 MB	8,196	—
Server C	**4**	**8,192 MB**	**0**	**✓ SELECTED**

WHY BEST-FIT?

- Maximises cluster utilisation — fills servers before opening new ones
- Deterministic and predictable — easy to reason about and verify with tests
- Simple to explain — lower score = tighter pack = better efficiency

REST API Endpoints

SERVERS

POST	/api/servers	Create a new server	201
GET	/api/servers	List all servers	200
GET	/api/servers/<id>	Get server by ID	200
PATCH	/api/servers/<id>/status	Update server status + evict	200
DELETE	/api/servers/<id>	Delete server + evict workloads	200

WORKLOADS

POST	/api/workloads	Create a new workload (PENDING)	201
GET	/api/workloads	List all workloads	200
GET	/api/workloads/<id>	Get workload by ID	200
PATCH	/api/workloads/<id>	Modify CPU / RAM allocation	200
DELETE	/api/workloads/<id>	Delete workload + free resources	200

ALLOCATIONS

POST	/api/allocations	Submit workload for allocation	201
GET	/api/allocations/<id>	Get allocation record by ID	200
GET	/api/health	Health check endpoint	200

MongoDB Collections & Domain Models

servers

name — string
Unique server identifier

cpu_capacity — int
Total CPU cores

ram_capacity — int
Total RAM in MB

status — enum
ONLINE / OFFLINE / MAINTENANCE

allocated_cpu — int
CPU currently in use

allocated_ram — int
RAM currently in use (MB)

workloads

name — string
Unique workload identifier

cpu_required — int
CPU cores needed

ram_required — int
RAM needed in MB

status — enum
PENDING / ALLOCATED

allocations

workload_id — ref
Reference to workload document

server_id — ref
Reference to server document

created_at — datetime
Timestamp of allocation

COMPUTED PROPERTIES ON SERVER MODEL

available_cpu = cpu_capacity − allocated_cpu

available_ram = ram_capacity − allocated_ram

cpu_utilisation_pct = (allocated_cpu / cpu_capacity) × 100

103 Tests. Zero Failures.

103	3	15	100%
Total Tests	Test Modules	Test Classes	Pass Rate

test_allocations.py	TestRule1OnlineOnly	ONLINE-only server filtering
	TestRule4BothResourcesMatter	CPU AND RAM both required
	TestRule5NoPartialAllocation	Atomicity — no partial writes
	TestRule6NoDuplicateAllocation	Concurrency guard
	TestRule7BestFitStrategy	Best-fit server selection
	TestRule8ResourceAccounting	Allocated counter accuracy
	TestServerStatusChangeEvictsWorkloads	Eviction on status change
	TestWorkloadDeletionFreesResources	Resource release on delete
	TestModifyAllocatedWorkloadResources	Delta capacity accounting
test_servers.py	TestCreateServer, TestUpdateServerStatus, TestDeleteServer	Server CRUD lifecycle
test_workloads.py	TestCreateWorkload, TestUpdateWorkloadResources, TestDeleteWorkload	Workload CRUD lifecycle

✓ 103 passed in 115.14s (0:01:55) — All green, every rule validated

09 SUMMARY

What was delivered

V1.1

Base Initialization

- Layered Flask architecture
- 10 allocation business rules
- Best-fit scheduling engine
- MongoDB Atlas integration
- Single-page frontend UI
- 58 passing tests

V1.2

Server Status & Workload Management

- Server status transitions (Online/Offline/Maintenance)
- Server deletion endpoint
- Automatic workload eviction with resource reset
- Compensating transaction pattern
- 93 passing tests

V1.3

Workload Management Enhancements

- Workload deletion with resource release
- Modify CPU/RAM of existing workloads
- Delta capacity accounting for allocated workloads
- Edit/Delete UI controls
- 103 passing tests

ENGINEERING PRINCIPLES APPLIED

Separation of Concerns

Routes / Service / Repository / Model — each layer has one job

Typed Error Model

Service raises exceptions; routes map them to HTTP codes

Integration Testing

Black-box tests over HTTP — no mocking, full-stack confidence

github.com/Sriram-Selvaperumal/tactive-assignment